

NS-FirmID: A Neuro-Symbolic Multi-Agent Framework for Reliable Firmware Version Identification at Internet Scale

Abstract—Reliable firmware version identification is paramount for Internet-scale vulnerability assessment. Existing methods struggle with banner heterogeneity, while promising large language model solutions are hindered by deployment resource and false positive issues. We introduce NS-FirmID, the first neuro-symbolic multi-agent framework tailored for robust firmware identification. By orchestrating Extraction, Verification, and Discrimination agents, our framework fuses neural semantic parsing with symbolic knowledge to enable gray-box uncertainty modeling and zero-shot generalization. sustains a 93% F1-score and remains resilient across diverse heterogeneity tiers. Notably, it reduces the false positive rate by over 17.1% compared to baseline SLMs, surpassing its 671B teacher model (DeepSeek-R1) while using only 1% of its parameters. Furthermore, in-the-wild deployment on 150,000 banners achieves coverage 211× broader than Nmap, revealing critical vulnerabilities in 27.4% of identified devices. Our framework is available at: <https://github.com/Anonymous-for-review930/NS-FirmID>.

Index Terms—Firmware Identification; Neuro-Symbolic AI; Multi-Agent Framework; Large Language Models; Internet Measurement; Vulnerability Assessment.

I. INTRODUCTION

In recent years, large-scale Internet measurement has emerged as a fundamental technique for understanding the Internet attack surface, assessing global device risk postures, and guiding patch deployment and asset management [1]. As the population of Internet-connected embedded and IoT devices continues to grow, identifying device attributes, particularly brands, models, and firmware versions, has emerged as a critical requirement for vulnerability assessment and network measurement [2]. Among these attributes, firmware version plays a uniquely important role: exploitation feasibility, patch availability, and vulnerability mappings almost always hinge on precise version information [3], [4].

Active probing remains one of the most scalable methodologies for Internet-wide device identification. By transmitting crafted packets and analyzing response banners, active probing provides immediate access to device-level information without relying on long-term traffic observation [5]. However, prior work has largely focused on coarse-grained attributes such as device type [6], brand [7], or model [8], [9]. Firmware version identification remains significantly under-explored because banners are highly heterogeneous, noisy, and often contain misleading text. As vividly illustrated in Fig. 1, real-world payloads routinely defeat traditional rule-based parsers. The challenges range from unreadable binary noise entangled with multiple confounding component versions (Example 1), to

verbose HTTP responses burying irregularly formatted version strings deep within HTML templates alongside misleading hardware metadata (Example 2). Distilling precise versions from such structural and semantic chaos is fundamentally challenging.

Example 1: Binary-Heavy and Noisy IoT Banner (HP Switch)

```
\x01\x00\x00\x9d\x02\x00\n\x00'\x12\x14\xdd\n\x02\x07
\x87\x02\x00\n\x02'\x13\x14\xdd\x00\xa8\n\x01\x01\x00
\x06\x00'\x12\x14\xdd\n\x00\x04\x00\x01\xde\x7f\x0b
\x00\x11...HP 1820-8G Switch J9979A, PT.01.10 , Linux
3.6.5-79c95a77, U-Boot 2012.10-00116-g3ab515c
(Jul 30 2014-10:52:01)...
```

Example 2: HTTP & HTML Mixed Banner (HP Printer)

```
HTTP/1.1 200 OK
Server:HP HTTP Server; HP OfficeJet 4650 series -
FlJ03A; Serial Number: TH583190280662; Built:Tue May 26,
2015 05:03:20PM { CUP1FN1522BR }
Content-Encoding: gzip
Content-Type: text/html
Last-Modified: Tue, 26 May 2015 17:03:20 GMT
...
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML
4.01//EN"> <html><head><meta http-equiv...
```

Fig. 1. Real-world banner examples defeating structural extraction. Example 1 mixes binary noise with confounding strings (e.g., OS and bootloader versions). Example 2 buries an uncommonly formatted target attribute within verbose HTTP/HTML payloads alongside misleading numerical strings (e.g., serial numbers). **Bold text** indicates confounding version-like strings, while **highlights** pinpoint target firmware versions.

Existing efforts fall into two categories: implicit-feature-based fingerprinting and explicit string extraction. Methods relying on implicit features (e.g., firmware image analysis [10], DOM structure modeling [11], web directory hashing [12], [13]) achieve high accuracy but fail to scale to Internet-wide settings due to practical barriers—firmware images are often inaccessible, encrypted, proprietary, or too costly to emulate. Conversely, explicit text-based approaches depend on manually maintained regular expressions or static fingerprint databases. Tools like Nmap [14] and Shodan [15] achieve high precision for known devices, but their coverage remains inherently constrained. The rapid expansion of IoT vendors, the prevalence of OEM/white-box devices, and the lack of standardized banner formats render such rule-based systems brittle, difficult to maintain, and ineffective for long-tail or unseen devices.

The emergence of Large Language Models (LLMs) offers a promising direction for interpreting unstructured network banners. Recent studies [2], [16], [17] have demonstrated the robust capabilities of LLMs in general device fingerprinting and label transfer. However, two fundamental barriers prevent current LLM-based approaches from reliably identifying firmware versions: 1) **The trade-off between deployment cost and model performance:** Large models incur significant inference cost, limiting their practicality for high-throughput measurement. Small Language Models (SLMs), while easier to deploy, lack deep domain knowledge and struggle to extract fine-grained attributes from noisy banners [18]. 2) **Lack of structured knowledge constraints:** While LLMs facilitate flexible attribute extraction via semantic reasoning, firmware versioning strictly adheres to vendor-specific, long-standing naming conventions and sequential patterns that are difficult for purely neural models to internalize. Without explicit knowledge injection, LLMs struggle to reliably capture these symbolic structures and the interdependencies between attributes. 3) **Hallucinations and entity confusion:** LLMs face inherent reliability challenges, specifically hallucinations and entity confusion. They tend to generate false positives with high confidence, rendering them unsuitable for direct application in measurement tasks requiring high fidelity [19]

These challenges raise the central question of this work:

Can we design a system that matches or surpasses LLM-level extraction accuracy, achieves the deployment efficiency of SLMs, and provides low false positive rates with reliable generalization to unseen firmware versions?

We present two key insights that guide our approach.

Insight 1: Firmware version identification exhibits a distinct combination of rich symbolic structure and high semantic noise. While neural models excel at parsing diverse banner formats, symbolic knowledge offers strong structural constraints and interpretability. Combining these two modalities enables a robust “extract–verify–decide” workflow that mirrors the reasoning process of human analysts.

Insight 2: While prompting a model to articulate its “confidence” can induce self-reflection regarding the correctness of its answers, LLMs suffer from inherent “hallucinations” and “overconfidence, verbalized confidence only from LLMs is insufficient for security tasks. Instead, reliable uncertainty estimation must integrate intrinsic signals that cannot be fabricated during inference (e.g., token-level log-probabilities) and deterministic external evidence (e.g., semantic stability in Chain-of-Thought (CoT) and alignment with symbolic knowledge). Such gray-box signals provide an actionable foundation for filtering hallucinations and reducing false positives.

Guided by these insights, we develop *NS-FirmID*, the first neuro-symbolic multi-agent framework for reliable firmware version identification at Internet scale. *NS-FirmID* orchestrates three complementary agents:

(1) **Extractor Agent (EA):** a fine-tuned Qwen2.5-7B [20] distilled from DeepSeek-R1 [21], responsible for semantic parsing of banners, balances semantic reasoning with deployment efficiency.

(2) **Validator Agent (VA):** a symbolic component that validates extracted attributes against a comprehensive knowledge base and leverages historical naming patterns to guide the *EA* through iterative refinement, thereby enabling generalization to unseen versions.

(3) **Discriminator Agent (DA):** a gray-box uncertainty model that fuses neural, semantic, and symbolic signals into a 28-dimensional feature vector that quantifies the overall confidence of candidates, thereby suppress false positives.

NS-FirmID synergizes the neural perception of LLMs with the symbolic rigor of structured device knowledge. This hybrid approach effectively navigates the resource-precision trade-off and mitigates the hallucination-induced false positives prevalent in LLM-based firmware identification. Our contributions are summarized as follows:

- We propose *NS-FirmID*, the first neuro-symbolic multi-agent framework tailored for Internet-scale firmware identification. By orchestrating an “extract–verify–decide” workflow, it unifies semantic extraction, symbolic verification, and confidence discrimination, achieving robust structure extraction from noisy banners.
- We design a gray-box uncertainty modeling approach and a pattern induction mechanism. Besides fusing intrinsic neural signals with symbolic evidence to rigorously mitigate false positives, our pattern induction facilitates the zero-shot identification of unseen version formats
- Evaluations on 5,185 real-world banners demonstrate that *NS-FirmID* significantly outperforms SLMs in false positives ($\downarrow 17.1\%$) and F1-score ($\uparrow 14.6\%$), even achieves performance exceeding the teacher model while using less than 1% of its parameters.
- We executed a large-scale measurement and vulnerability assessment in the wild. Our deployment on 150,000 banners yielded firmware versions coverage surpassing Nmap by 211 times. Subsequent vulnerability mapping revealed that 27.4% of the identified CVEs are of Critical severity, effectively transforming raw asset discovery into actionable risk intelligence.

RoadMap. Section II outlines the background of our work. Section III presents the methodology. Section IV details the real-world experiments and firmware version status in the wild. Section V discusses limitations, and Section VI concludes.

II. BACKGROUND

Network asset discovery fundamentally relies on either passive traffic analysis or active probing. We prioritize active probing because passive methods are highly sensitive to environmental noise and often fail to distinguish firmware versions that share identical traffic signatures. Early active probing research primarily targeted coarse-grained attributes, such as device types [6], brands [7], [22], or models [8]. To optimize extraction, tools like ARE [9] and Devtag [23] developed automated rule generation engines and integrated manually crafted fingerprints from repositories like Nmap [14] and ZTag [24].

Recently, Large LLMs have been adopted for device fingerprinting. BLMPProbe [2] utilizes LLM-guided label exploration for cross-port fingerprinting. Sarabi et al. [16] and Mahmood-den et al. [17] leverage instruction fine-tuning and information theory to generate high-confidence pseudo-labels from internet scans. However, these pseudo-labels can inadvertently amplify hallucinations and erroneous associations. More importantly, these efforts remain confined to manufacturer- or model-level identification, leaving the more complex challenge of fine-grained firmware version extraction largely unresolved.

Current research on firmware version identification generally follows two paradigms: implicit feature generation and plaintext keyword extraction.

A. Implicit Feature Generation

While Nmap [14] pioneers active OS fingerprinting via TCP/IP stack anomalies, its reliance on kernel-level differences yields overly coarse results, as disparate IoT devices frequently share common Linux kernels. To achieve finer granularity, researchers analyze internal firmware structures. Costin et al. [10] and Zhang et al. [11] apply machine learning and graph neural networks to firmware binaries, while Li et al. [12] and Xie et al. [13] construct fingerprints from filesystem hashes and optimized probing sequences. Despite their precision, these methods require access to the physical firmware images, which are increasingly proprietary, encrypted, or difficult to emulate. This heavy reliance on complex feature engineering fundamentally limits their viability for Internet-scale measurement [1].

B. Plaintext Keyword Extraction

Explicit text extraction directly parses version strings from active probing banners or certificates. Mainstream tools like Nmap [14] and ZTag [24], as well as search engines like Shodan [15] and Censys [25], rely on static regular expressions or hardcoded fingerprint databases. While highly accurate for known devices, they suffer from a severe long-tail problem: static rules cannot scale to cover the massive influx of OEM and white-box IoT devices.

Recent measurement studies have attempted to optimize this process. Liang et al. [26] applied reinforcement learning for multi-protocol probing, while Yu et al. [27] leveraged weak credentials to extract versions from management interfaces—a tactic increasingly deprecated by modern security practices. Notably, Ebbers [1] conducted a massive empirical study of IoT firmware deployments using extensive regex rulesets. However, purely regex-based extraction lacks semantic awareness. It is notoriously brittle: overly strict rules lead to low recall, whereas broad rules incur high false-positive rates.

Summary. Despite prior efforts, existing plaintext and implicit feature methods fundamentally rely on predefined static signatures or rigid rules. Ultimately, maintaining such rules for thousands of heterogeneous, non-standard, and obfuscated banner formats proves to be a fragile and unscalable paradigm. This crucial gap necessitates a novel approach that blends

semantic understanding with structured verification, driving the design of NS-FirmID.

III. METHOD

A. Problem Definition and System Overview

The goal of *NS-FirmID* is to accurately identify a device’s firmware version from noisy banner strings, along with its corresponding brand and model. Given the raw banner text $S \in \mathcal{S}$ obtained from active probing, our objective is to extract a device attribute triplet (b, m, v) , where b denotes the brand, m denotes the model, and v denotes the firmware version. Formally, we define the attribute extraction task as:

$$\mathcal{F} : \mathcal{S} \rightarrow (b, m, v), \quad b \in \mathcal{B}, m \in \mathcal{M}, v \in \mathcal{V}$$

where $\mathcal{B}$, $\mathcal{M}$, and $\mathcal{V}$ denote the semantic spaces for the brand, model, and version, respectively.

As shown in Fig. 2, *NS-FirmID* operates through three synergistic agents. The **Extractor Agent (EA)** utilizes a fine-tuned SLM to parse noisy banners into candidate triplets alongside their CoT reasoning paths. **To defend against fabricated formats and structural hallucinations, these outputs are scrutinized by the Validator Agent (VA), which leverages a symbolic knowledge base for rigorous verification and guides a secondary extraction via version pattern induction.** To ensure high fidelity, the **Discriminator Agent (DA)** functions as a gray-box reliability estimator, fusing internal neural signals with external symbolic evidence into a 28-dimensional feature vector to rigorously quantify confidence.

The inference pipeline follows a progressive “Extract-Verify-Decide” workflow. Specifically, the process initiates with the EA’s preliminary extraction. The VA then validates these findings; if the result is incomplete or verified as uncertain, it triggers a feedback loop for knowledge-injected refinement. Finally, the DA evaluates the aggregated neuro-symbolic evidence, accepting only those attributes that satisfy a strict confidence threshold.

B. Extractor Agent

To strike a balance between deployment cost and performance, we leveraged knowledge distillation to transfer the powerful reasoning capabilities of a Teacher Model to a local Student Model via Supervised Fine-Tuning (SFT). Specifically, we guided the Teacher Model (DeepSeek-R1) to generate detailed and logically clear CoT reasoning traces, structuring the process as ‘Keyword Judgment → Evidence Extraction → Confidence Scoring Details’ through meticulously designed prompts. To improve the quality of the CoT, we performed two rounds of generation through the self-optimization of the teacher model. These high-quality CoT traces were then utilized to fine-tune the Student SLM (Qwen2.5-7B [20]).

We employ LoRA [28] for parameter-efficient fine-tuning. To strengthen the model’s inherent CoT reasoning capability,

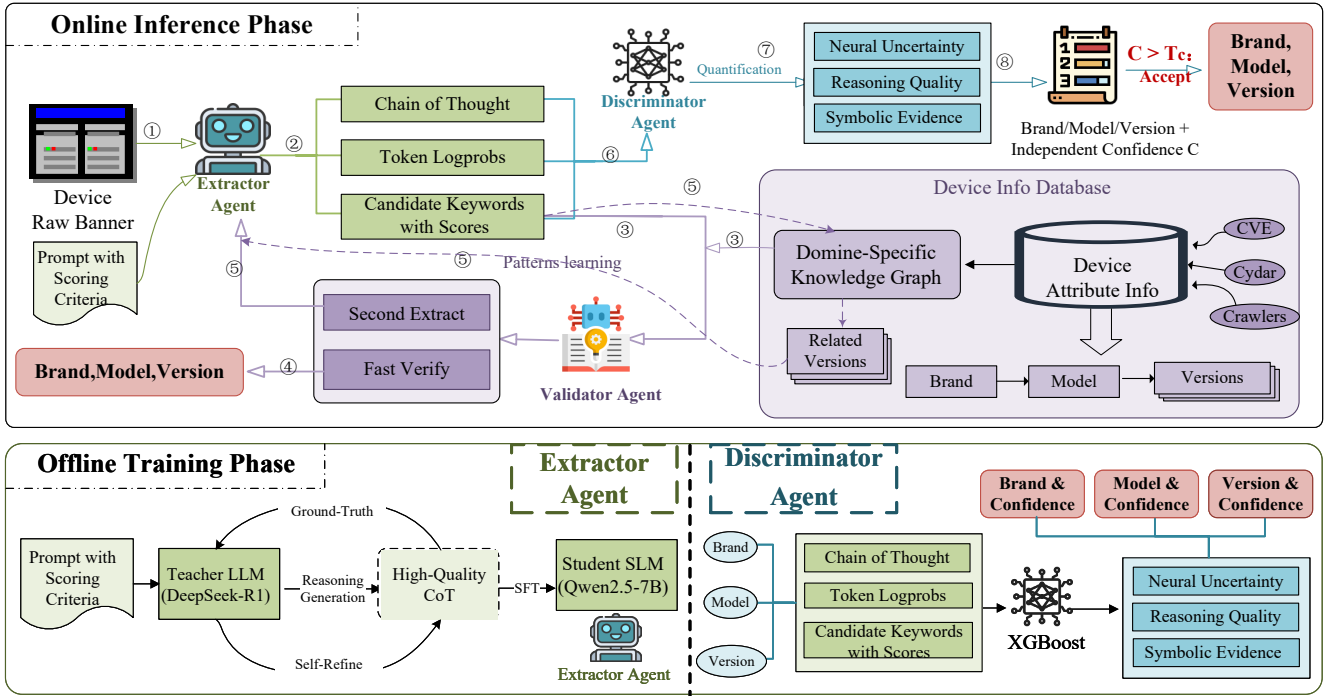


Fig. 2. The architecture of *NS-FirmID*, a neuro-symbolic multi-agent framework for reliable firmware identification from noisy Internet banners. *NS-FirmID* integrates neural extraction (EA), symbolic verification (VA), and confidence fusion (DA), leveraging structured knowledge and grey-box uncertainty reasoning to reduce false positives and generalize to unseen version formats.

we apply a higher weight to the reasoning component $\mathcal{T}$ within the loss function:

$$\mathcal{L}_{\text{SFT}}(\theta) = \frac{1}{N} \sum_{i=1}^N [\lambda_{\mathcal{T}} \cdot \mathcal{L}_{\mathcal{T}}^i + \lambda_{\mathcal{E}} \cdot \mathcal{L}_{\mathcal{E}}^i + \lambda_{\mathcal{C}} \cdot \mathcal{L}_{\mathcal{C}}^i]$$

where $\mathcal{L}_{\mathcal{T}}^i$ denotes the loss for the reasoning component, $\mathcal{L}_{\mathcal{E}}^i$ denotes the loss for the extraction result component, and $\mathcal{L}_{\mathcal{C}}^i$ denotes the loss for the confidence analysis component. This weighting strategy explicitly encourages the model to focus on learning the underlying reasoning process, rather than merely memorizing input-output mappings.

For a given banner S_i , the initial extraction $\hat{y}_{\text{EA}}(S_i)$ yielded by the EA comprises three vital artifacts:

- **Structured JSON with Confidence Scores:**

$$\begin{aligned} \text{EA}_{\mathcal{E}}^{(1)}(S_i) &= \{ \text{"Brand"} : \{K_{\text{Brand}}^{(1)}, C_{\text{Brand}}^{(1)}\}, \\ &\quad \text{"Model"} : \{K_{\text{Model}}^{(1)}, C_{\text{Model}}^{(1)}\}, \\ &\quad \text{"Firmware"} : \{K_{\text{Fw}}^{(1)}, C_{\text{Fw}}^{(1)}\} \end{aligned}$$

where K represents the extracted keyword for each attribute, and C is its corresponding subjective confidence score.

- **CoT Reasoning Text:** $\text{EA}_{\mathcal{T}}^{(1)}(S_i) = [r_b^{(1)}, r_m^{(1)}, r_v^{(1)}]$.
- **Token-Level Log Probability Sequence:**
 $\text{EA}_{\mathcal{C}}^{(1)}(S_i) = \{\log p_{\theta}(\text{token}_t \mid \text{prefix})\}_{t=1}^T$

C. Validator Agent

Given the severe noise and format heterogeneity in real-world banners, the EA may occasionally miss idiosyncratic

version formats specific to certain vendors. Moreover, when adversarial decoys or confounding strings disrupt the extraction, the EA's internal reasoning often struggles to reliably detect false positives. This fundamental limitation necessitates external symbolic verification.

To anchor the neural extraction, the VA maintains a hierarchical device knowledge base $\mathcal{K}$ (detailed in Section IV-A), formally structured as an attribute association graph:

$$\mathcal{K} = \{b \mapsto \{m \mapsto \{v_1, v_2, \dots\}\}\}$$

where b represents a brand, m represents a model under b , and each v_i represents a verified firmware version string associated with m .

To query this knowledge base against noisy predictions ($\text{EA}_{\mathcal{E}}$), the VA employs a hierarchical fuzzy matching algorithm (Algorithm 1). The process progressively narrows the search space from brand $\rightarrow$ model $\rightarrow$ version. Initially, brand names are normalized (lowercased and stripped of special characters), and the Edit Distance similarity is calculated:

$$\text{sim}_b(b_c, b_k) = 1 - \frac{\text{EditDist}(\text{norm}(b_c), \text{norm}(b_k))}{\max(|b_c|, |b_k|)}$$

Subsequently, we conduct model matching among candidates belonging to the matched brand. Because model structures are semantically richer and prone to variations, we employ a hybrid similarity metric that combines sequence matching and semantic profiling:

$$\text{sim}_m(m_c, m_k) = \alpha \cdot \text{sim}_{\text{str}}(m_c, m_k) + \beta \cdot \text{sim}_{\text{sem}}(m_c, m_k)$$

Here, the string similarity sim_{str} is calculated using Sequence-Matcher (SM), supplemented with an extra bonus for substring inclusion:

$$\text{sim}_{\text{str}}(m_c, m_k) = \text{SM}(m_c, m_k) + \lambda \cdot \mathbf{1}[m_c \subseteq m_k \vee m_k \subseteq m_c]$$

Meanwhile, the semantic similarity sim_{sem} evaluates the structural components of the model identifier:

$$\text{sim}_{\text{sem}}(m_c, m_k) = w_1 \cdot \delta_{\text{line}} + w_2 \cdot \text{sim}_{\text{gen}} + w_3 \cdot \text{sim}_{\text{spec}}$$

where δ_{line} is the product line indicator, $\text{sim}_{\text{gen}} = \max(0, 1 - |g_c - g_k| \times 0.2)$ measures generation proximity, and sim_{spec} assesses the specification matching degree (e.g., Pro, Max, Mini).

Finally, the version matching process is executed. It attempts an exact normalization match first; if unsuccessful, fuzzy sequence matching is employed as a fallback:

$$\text{sim}_v(v_c, v_k) = \begin{cases} 1.0 & \text{if } \text{norm}(v_c) = \text{norm}(v_k) \\ \text{SM}(v_c, v_k) & \text{otherwise} \end{cases}$$

Given a matched reference (b_k, m_k, v_k) and their respective similarities (s_b, s_m, s_v) , the comprehensive validation confidence score is computed as a weighted fusion:

$$\sigma_{\text{val}} = w_b \cdot s_b + w_m \cdot s_m + w_v \cdot s_v$$

where we specify $w_b = 0.2$, $w_m = 0.3$, $w_v = 0.5$ to assign the highest priority to firmware version accuracy. If σ_{val} indicates a strict alignment with $\mathcal{K}$, the identification process successfully terminates.

However, if the initial verification fails, or if the EA identifies a valid brand-model pair but fails to extract a highly obfuscated firmware version, the VA triggers a knowledge-guided secondary extraction. Specifically, it aggregates the historical version lists from the matched model candidates $\mathcal{M}_{\text{matches}} = \{(m_k^{(i)}, s_m^{(i)}, \mathcal{V}_{m_k}^{(i)})\}_{i=1}^K$. By injecting these known patterns alongside the raw banner S_i and the preliminary prediction $\hat{y}_{\text{EA}}^{(1)}(S_i)$ back into the EA's prompt, the VA establishes an in-context learning loop. This dynamically induces the EA to comprehend vendor-specific naming semantics, enabling it to produce a refined extraction $\hat{y}_{\text{VA}}^{(2)}(S_i)$ without requiring expensive model retraining.

This hierarchical strategy significantly enhances the robustness of *NS-FirmID* against massive, diverse data streams. Notably, it also enables the system to inversely infer a missing brand directly from a distinctive model identifier, which proves particularly effective when the raw banner lacks explicit brand keywords.

D. Discriminator Agent

To mitigate the intrinsic hallucination and overconfidence issues prevalent in LLM-based extractions, the DA functions as a gray-box uncertainty estimator. Unlike existing methods that rely solely on the model's verbalized confidence (which is notoriously poorly calibrated), the DA acts as a comprehensive reliability estimator. It fuses unforgeable internal neural signals

Algorithm 1 Hierarchical Fuzzy Matching Algorithm

Require: Candidate (b_c, m_c, v_c) , KB $\mathcal{K}$

Ensure: $\mathcal{V}_{\text{results}}, \mathcal{M}_{\text{matches}}$

```

1:  $\mathcal{V}_{\text{results}} \leftarrow \emptyset, \mathcal{M}_{\text{matches}} \leftarrow \emptyset, \mathcal{B}_{\text{matched}} \leftarrow \emptyset$ 
   Stage 1: Brand Matching
2: for  $b_k \in \text{keys}(\mathcal{K})$  do
3:    $s_b \leftarrow 1 - \frac{\text{EditDist}(\text{norm}(b_c), \text{norm}(b_k))}{\max(|b_c|, |b_k|)}$ 
4:   if  $s_b \geq 0.8$  then
5:      $\mathcal{B}_{\text{matched}} \leftarrow \mathcal{B}_{\text{matched}} \cup \{(b_k, s_b)\}$ 
6:   end if
7: end for
   Stage 2: Model Matching
8: for  $(b_k, s_b) \in \mathcal{B}_{\text{matched}}$  do
9:   for  $m_k \in \text{keys}(\mathcal{K}[b_k])$  do
10:     $s_m \leftarrow \text{HybridSim}(m_c, m_k)$ 
11:    if  $s_m < 0.7$  then continue
12:    end if
13:     $V_m \leftarrow \mathcal{K}[b_k][m_k]$ 
14:     $\mathcal{M}_{\text{matches}} \leftarrow \mathcal{M}_{\text{matches}} \cup \{(m_k, s_m, V_m)\}$ 
   Stage 3: Version Matching
15:   for  $v_k \in V_m$  do
16:      $s_v \leftarrow \text{VerSim}(v_c, v_k)$ 
17:     if  $s_v \geq 0.8$  then
18:        $\sigma_{\text{val}} \leftarrow 0.2s_b + 0.3s_m + 0.5s_v$ 
19:        $\mathcal{V}_{\text{results}} \leftarrow \mathcal{V}_{\text{results}} \cup \{(b_k, m_k, v_k, \sigma_{\text{val}})\}$ 
20:     end if
21:   end for
22: end for
23: end for
24:  $\mathcal{V}_{\text{results}} \leftarrow \text{top-k}(\mathcal{V}_{\text{results}}, 5)$ 
25:  $\mathcal{M}_{\text{matches}} \leftarrow \text{top-k}(\mathcal{M}_{\text{matches}}, 5)$ 
26: return  $\mathcal{V}_{\text{results}}, \mathcal{M}_{\text{matches}}$ 

```

with external symbolic validation to rigorously quantify the trustworthiness of each extracted attribute.

Because the semantic spaces and inherent extraction difficulties vary across attributes, a dedicated DA is trained independently for brand, model, and firmware version. We frame this reliability estimation as a binary classification task. For each candidate attribute generated by the EA and VA, DA constructs a 28-dimensional feature vector $\mathbf{x} \in \mathbb{R}^{28}$ to predict a reliability score $P(y = 1|\mathbf{x})$. As detailed in Table I, the feature space spans three core analytical dimensions:

1) Neural Uncertainty Quantification: LLM hallucinations typically manifest as entropy spikes during token generation. We extract token-level log-probabilities $\mathcal{L} = \{\log p(t_i|t_{<i})\}_{i=1}^N$ from EA's inference trace to compute intrinsic model uncertainty metrics. Specifically, we calculate the Perplexity (PPL) to measure the model's overall decoding confusion:

$$\text{PPL}(\mathbf{t}) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(t_i|t_{<i}) \right)$$

Furthermore, we compute the Low-Confidence Ratio r_{low} ,

TABLE I
FEATURE SET FOR THE DISCRIMINATOR AGENT

Feature Group	Dims	Description / Components
1. Log Probability Features (8 Dims) → Neural Uncertainty Signals		
Logprob Statistics	4	$\bar{l}, \min l, \max l, \sigma_l$: Statistical measurements of log probability
Perplexity (PPL)	1	PPL: Perplexity ($\exp(-\bar{l})$)
Low-Confidence Ratio	1	r_{low} : Ratio of low-confidence tokens
Logprob Range	1	$\Delta l = \max l - \min l$: Log probability range
2. Semantic Features (5 Dims) → Reasoning Quality		
Keyword Counts	3	$n_{\text{certain}}, n_{\text{uncertain}}, n_{\text{negative}}$: Keyword counts
Reasoning Length	1	$ \mathcal{T}_{\text{reasoning}} $: Length of the reasoning text
Reasoning Tokens	1	n_{token} : Number of reasoning tokens
3. Structural Features (8 Dims) → Reasoning Quality		
Has Value	1	$\mathbf{1}[\text{text}_{\text{has_value}}]$: Whether a value has been extracted
Extracted Length	1	$ v_{\text{extracted}} $: Extracted value length
Word Count	1	n_{words} : Number of words
Format Features	2	$\mathbf{1}[\text{text}_{\text{has_number}}], \mathbf{1}[\text{text}_{\text{has_special}}]$
Generic Term	1	$\mathbf{1}[\text{text}_{\text{is_generic}}]$: Whether it is a generic term
Too Short	1	$\mathbf{1}[\text{text}_{\text{is_too_short}}]$: Whether it is too short
4. Consistency Features (3 Dims) → Symbolic Evidence		
Confidence Scores	2	$c_{\text{claimed}}, c_{\text{recommended}}$: Claimed and recommended confidence scores
Confidence Difference	1	$\Delta c = c_{\text{claimed}} - c_{\text{recommended}} $: Confidence score difference
5. Attribute-Specific Features (2 Dims) → Symbolic Evidence		
Attribute Heuristics	2	Heuristic features designed for different attribute characteristics
6. Validation Features (2 Dims) → Symbolic Evidence		
Validation Score	1	σ_{val} : Validation confidence score
Validation Success	1	$\mathbf{1}[\text{validated}]$: Whether validation passed

defined as the proportion of tokens with probabilities falling below a strict threshold τ (e.g., e^{-5}), which serves as a highly sensitive proxy for localized decoding uncertainty.

2) Confidence Consistency & Calibration: A fundamental systemic innovation of *NS-FirmID* is detecting the “Calibration Gap” between the model’s explicitly claimed confidence and its actual underlying generation probability. We define two contrasting confidence metrics:

- **Claimed Confidence (C_{claim}):** The subjective confidence score explicitly verbalized by the LLM within its structured JSON output.
- **Recommended Confidence (C_{rec}):** An objective confidence metric aggregated from neural and semantic signals. Formally, C_{rec} is a weighted fusion of the geometric mean of token probabilities (S_{prob}), semantic certainty derived from keywords (S_{sem}), and perplexity stability (S_{ppl}):

$$C_{\text{rec}} = \alpha \cdot S_{\text{prob}} + \beta \cdot S_{\text{sem}} + \gamma \cdot S_{\text{ppl}}$$

where $S_{\text{prob}} = \exp\left(\frac{1}{N} \sum_{l \in \mathcal{L}} l\right)$ represents the raw generation probability.

Based on these contrasting metrics, we derive the Confidence Gap feature: $\Delta C = |C_{\text{claim}} - C_{\text{rec}}|$. A massive ΔC typically exposes severe “overconfidence,” where the model claims high certainty despite exhibiting weak underlying probabilities—a definitive physical signature of hallucination.

3) Structural & Semantic Verification: We extract structural features $\mathbf{x}_{\text{struct}}$ to enforce strict formatting constraints (e.g., `has_special` checks for version-specific characters like ‘.’ or ‘-’). Additionally, we analyze the CoT trace to count specific uncertainty keywords (e.g., “might”, “unclear”) against definitive certainty keywords (e.g., “explicitly states”). This quantifies the semantic stability of the reasoning process, allowing the DA to safely abort the extraction when the EA is overwhelmed by adversarial decoys or conflicting strings.

4) Decision Model: The fused feature vector $\mathbf{x}$ is fed into an XGBoost [29] classifier, trained to minimize the binary cross-entropy loss $\mathcal{L}_{\text{DA}}$:

$$\mathcal{L}_{\text{DA}} = - \sum_i [y_i \log f(\mathbf{x}_i) + (1 - y_i) \log(1 - f(\mathbf{x}_i))]$$

During inference, an attribute is accepted only if its probability exceeds a threshold T_c . Crucially, T_c serves as a tunable operating point to balance the recall-FPR trade-off based on deployment scenarios. For instance, broad asset discovery may relax T_c to maximize coverage, whereas critical vulnerability matching demands a conservative, high T_c to safely abort ambiguous extractions and retain only high-fidelity intelligence. Ultimately, this adaptive and lightweight filtering mechanism ensures *NS-FirmID* remains robust and practically scalable for Internet-wide measurements.

IV. EVALUATION

In this section, we rigorously evaluate the performance of *NS-FirmID* using real-world Internet banners. Our evaluation is driven by the following research questions:

- **RQ1 (Distillation & Efficacy):** Can *NS-FirmID* achieve a superior precision and recall balance compared to state-of-the-art baselines, and successfully match the performance of its resource-intensive teacher LLM?
- **RQ2 (Trustworthiness):** Can *NS-FirmID* effectively filter out hallucinations and overconfident predictions without severely degrading extraction recall?
- **RQ3 (Ablation Study):** How do the components (EA, VA, DA) contribute to the overall pipeline, and which features dominate the DA’s decisions?
- **RQ4 (In-the-Wild Generalization):** Can *NS-FirmID* maintain robustness on heterogeneous Internet-scale data, and how does this fine-grained visibility empower downstream vulnerability assessments?

A. Datasets

To evaluate *NS-FirmID* under realistic deployment conditions, we constructed a comprehensive dataset integrating active probing traces with a verified firmware knowledge base. The dataset statistics are summarized in Table II.

TABLE II
DATASET STATISTICS

Category	Split	Samples	B	M	V	P
KG	CPE	-	1.3k	10k	41k	-
	Cydar	-	1.6k	104k	267k	-
	Crawled	-	14	16k	180k	-
	Total	-	2.7k	119k	256k	-
Labeled	Train	1925	987	533	482	51
	Test	5185	528	1545	1766	69
Unlabeled	-	150k	-	-	-	94

Note: B, M, V, P denote Brand, Model, Firmware Version, and Protocol.

Knowledge Base (K): We constructed a hierarchical device attributes database by aggregating data from: 1) The official CPE [4] dictionary, 2) Cydar [30], an IoT asset intelligence platform, and 3) Custom crawlers targeting vendor firmware release archives. This knowledge base serves as the symbolic ground truth for VA’s validation and In-Context Learning.

Labeled Evaluation Corpus: We scanned over 100 millions IPv4 hosts using public sources [31], [15], [32], [30]. All scans strictly followed ethical guidelines and opt-out policies. Responses were deduplicated, parsed via Cydar, and curated by three IoT security experts.

Given the prohibitive cost of Internet-scale firmware annotation, our evaluation specifically tests data-efficient generalization. We initially partitioned labeled data into training and testing candidate splits. From the training pool, we strictly selected instances where the Teacher Model generated high-quality CoT trajectories to serve as our final SFT set. To rigorously challenge our model and prevent data leakage,

remaining unselected training samples were migrated into the testing split, forming an expansive evaluation set. As Fig. 3 depicts, the testing set exhibits massive cardinality across models and versions (secondary logarithmic axis), confirming the extreme diversity of our evaluation corpus.

Unlabeled In-the-Wild Dataset: To assess real-world scalability, we compiled a 150k-sample corpus, collected identically but intentionally left unverified. This evaluates the system’s robust generalization and throughput on raw Internet data.

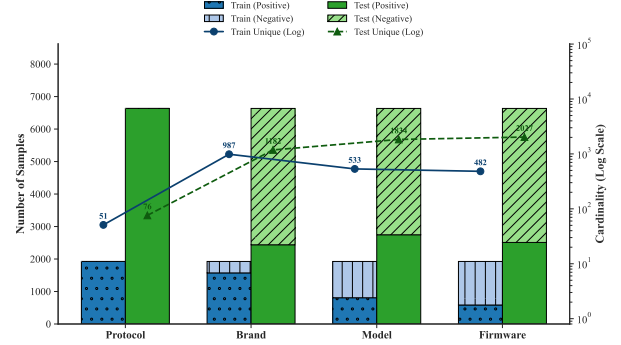


Fig. 3. Comparison of training and testing datasets across four attributes. Stacked bars (primary y-axis) show sample volume. Trend lines (secondary logarithmic y-axis) indicate cardinality (unique value count).

B. Evaluation Metrics

Following standard practices, we report Precision, Recall, F1-score, Accuracy, and False Positive Rate (FPR):

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{FPR} = \frac{FP}{FP + TN}, \quad \text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

C. Baselines

We benchmark *NS-FirmID* against representative traditional rule-based tools and modern LLMs:

Nmap [14]: The industry-standard open-source network scanner, relying on an internal rule base of over 3,000 regular expressions.

Shodan [15]: A global device search engine that utilizes manually curated signatures to extract firmware attributes.

DeepSeek-R1 [21]: A state-of-the-art massive LLM (671B), serving as the teacher model for our EA.

Qwen-14B [33], Qwen2.5-7B [20], Llama-3-8B [34]: Representative SLMs with varying architectures and parameter scales, evaluating zero-shot semantic extraction capabilities.

D. Overall Performance (RQ1)

To evaluate effectiveness, we benchmark *NS-FirmID* against the baselines detailed in Section IV-C. Table III presents a comprehensive performance comparison across Brand, Model,

TABLE III
PERFORMANCE COMPARISON OF STATE-OF-THE-ART METHODS

Method	Brand			Model			Firmware Version		
	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall
Nmap	0.013	0.839	0.006	0.014	0.886	0.007	0.003	1.000	0.001
Shodan	0.166	0.312	0.112	0.036	0.060	0.025	0.062	0.606	0.033
Qwen2.5-7B	0.514	0.473	0.716	0.667	0.615	0.844	0.784	0.727	0.960
Llama-3-8B	0.508	0.381	0.760	0.621	0.551	0.712	0.660	0.579	0.768
Qwen-14B	0.480	0.320	0.963	0.594	0.435	0.936	0.670	0.507	0.987
<i>NS-FirmID</i> (7B)	0.734	0.713	0.924	0.805	0.785	0.796	0.930	0.927	0.887
DeepSeek-R1 (671B)	0.709	0.695	0.732	0.774	0.735	0.862	0.855	0.816	0.983

and Firmware Version extraction. Traditional rule-based approaches struggle in the heterogeneous IoT ecosystem; for instance, Nmap achieves near-perfect precision via rigid regular expressions but suffers a catastrophic lack of recall (0.001 for firmware versions), proving static signatures cannot scale to diverse in-the-wild banners. Conversely, while standard SLMs like Qwen-14B demonstrate robust semantic understanding with a remarkable 0.987 recall, their lack of domain-specific verification leads to over-interpreting noise as valid attributes, yielding poor precision (0.507). *NS-FirmID* effectively bridges this gap by synergizing neural extraction with symbolic verification, achieving an impressive F1-score of 0.930 and surpassing the best SLM baseline (Qwen2.5-7B) by an absolute margin of 14.6%.

More notably, our 7B model outperforms its 671B teacher model (DeepSeek-R1) across key metrics, despite utilizing less than 1% of its parameters. This seemingly counterintuitive result is a direct testament to the efficacy of our domain-specific neuro-symbolic distillation. Through meticulous prompt engineering, we curated a high-quality CoT dataset tailored exclusively for banner extraction. Consequently, while the massive, general-purpose teacher model occasionally drifts into verbose or irrelevant reasoning when confronted with chaotic banner noise, our EA is strictly fine-tuned to precisely isolate device-specific semantic patterns. Furthermore, potential extraction errors generated by the EA are systematically mitigated by the subsequent symbolic verification phases of the VA, yielding a highly specialized and accurate inference pipeline.

Ultimately, although explicitly designed for firmware version extraction, *NS-FirmID* jointly leverages brand and model identification. This interconnected approach maintains robust version inference even when auxiliary attributes are sparse or missing from the raw banner, effectively overcoming the intrinsic fragility of isolated extraction methods.

Answer to RQ1: *NS-FirmID* achieves a superior F1-score of **0.930**, overcoming static rules’ low recall (Nmap: 0.001) and SLMs’ poor precision (Qwen-14B: 0.507). Remarkably, it outperforms its 671B teacher (F1: 0.855) using < 1% of the parameters.

E. Ablation Study (RQ2 & RQ3)

To rigorously isolate the contribution of each component and evaluate the system’s reliability calibration, we conduct a stepwise ablation study. Table IV summarizes the performance trajectory as we incrementally integrate the EA, VA, and DA.

Impact of Neuro-Symbolic Components. The standard supervised fine-tuning (EA only) establishes a robust semantic baseline (F1 = 0.822). However, relying exclusively on the fine-tuned SLM yields a FPR of 14.7%. This indicates that while the model learns to extract plausible version-like strings, it inherently struggles to distinguish valid firmware versions from stochastic hallucinations triggered by chaotic banner payloads.

Integrating the VA introduces a strict external symbolic constraint, actively rejecting candidates that violate vendor-specific formatting rules or historical naming patterns. This step yields a critical precision gain, boosting the metric from 0.779 to 0.843 and suppressing the FPR to 9.6%.

However, the most decisive performance leap occurs with the deployment of the DA. By fusing intrinsic neural uncertainty with objective symbolic evidence, the DA effectively neutralizes overconfidence, driving the FPR down to a minimal 4.1% while elevating the overall F1-score to 0.930. Ultimately, compared to the zero-shot base model, the complete *NS-FirmID* pipeline achieves an absolute FPR reduction of 17.1% and an F1 enhancement of 14.6%, gracefully trading a nominal drop in recall for highly reliable precision.

Answer to RQ2: *NS-FirmID* effectively filters hallucinations and neutralizes overconfident predictions, driving the FPR down to a minimal 4.1%. Crucially, it avoids severely degrading recall, trading a mere 7.3% absolute drop for an overwhelming 20.0% precision surge.

Reliability Calibration and Uncertainty Modeling. A core hypothesis of our framework is that intrinsic neural uncertainty serves as a highly reliable proxy for detecting hallucinations. We validate this through a feature importance analysis of the DA during firmware version extraction (Fig. 4). Among the 28-dimensional feature set, PPL and the Confidence Gap ΔC (defined in Section III-D) emerge as the most dominant predictors. This compellingly confirms that when the SLM hallucinates or

TABLE IV
ABLATION METRICS SUMMARY FOR FIRMWARE VERSION
(VALUES IN PARENTHESES INDICATE ABSOLUTE PERCENTAGE POINT CHANGES COMPARED TO THE QWEN2.5-7B BASE MODEL)

Strategy	F1	Precision	Recall	Accuracy	FPR
Base Model: Qwen2.5-7B	0.784	0.727	0.960	0.852	0.212
EA only	0.822 (↑3.80%)	0.779 (↑5.20%)	0.897 (↓6.30%)	0.869 (↑1.70%)	0.147 (↓6.50%)
EA+VA	0.871 (↑8.70%)	0.843 (↑11.6%)	0.895 (↓6.50%)	0.901 (↑4.90%)	0.096 (↓11.6%)
EA+VA+DA (<i>NS-FirmID</i>)	0.930 (↑14.6%)	0.927 (↑20.0%)	0.887 (↓7.30%)	0.933 (↑8.10%)	0.041 (↓17.1%)
Teacher Model: DeepSeek-R1	0.855	0.816	0.983	0.897	0.169

generates ungrounded predictions, it consistently exhibits measurable entropy spikes and severe calibration errors that the DA can effectively intercept. Furthermore, structural features, such as `has_special`, also play a pivotal role, indicating that strict syntax compliance remains a strong prerequisite for extraction validity.

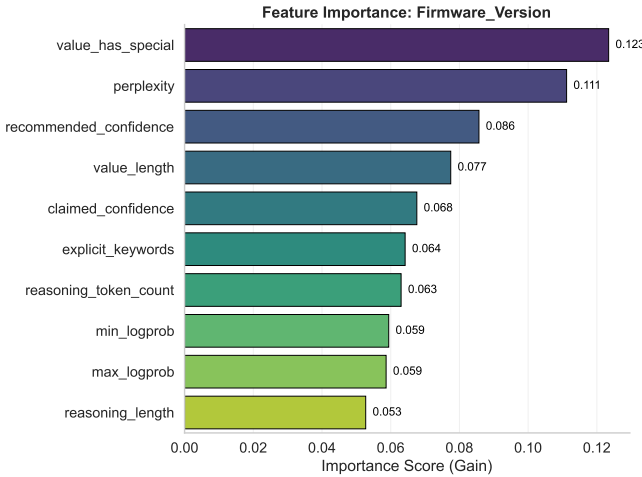


Fig. 4. DA feature importance for firmware version extraction. Neural uncertainty metrics (Perplexity and Confidence Gap) dominate, validating their efficacy in hallucination detection.

Regularization of Reasoning Dynamics. We further investigate how our domain-specific SFT alters the model’s inferential behavior by visualizing the correlation between input banner length and output CoT token length (Fig. 5). The zero-shot base model (blue) exhibits a highly entropic distribution, frequently hitting the 4,000-token generation ceiling even for short inputs. This pattern is indicative of “hallucination loops,” where the unaligned model falls into circular reasoning until forcefully truncated, introducing unpredictable inference latency. Conversely, the SFT-optimized EA (orange) demonstrates a tightly bounded, horizontal distribution, consistently generating 400–800 CoT tokens regardless of input length. This structural regularization indicates the EA has internalized a deterministic schema (“Extract, Verify, Conclude”), effectively mitigating the stochastic over-reasoning typical of zero-shot SLMs.

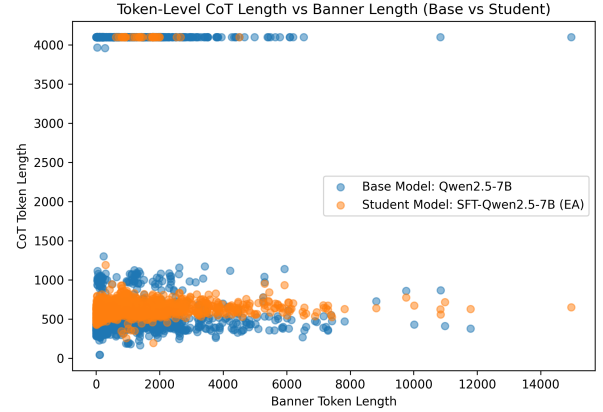


Fig. 5. Correlation between input banner and output CoT token lengths. The SFT-optimized EA (orange) effectively suppresses the stochastic “hallucination loops” prevalent in the zero-shot base (blue), yielding predictable inference latency.

Answer to RQ3: Each component is indispensable: the EA provides regularized semantic extraction, the VA enforces strict symbolic compliance, and the DA meticulously filters overconfident errors. Crucially, the DA’s decisions are predominantly driven by intrinsic neural uncertainty (Perplexity and Confidence Gap). This unified neuro-symbolic synergy ultimately yields a maximal F1 (93%) and a minimal FPR (4.1%).

F. Generalization and Application in the Wild (RQ4)

To evaluate the scalability and practical utility of our framework, we deployed *NS-FirmID* on a dataset of 150,000 unlabeled banners collected from the public Internet and initially filtered by Cydar. This “in-the-wild” experiment rigorously assesses the system’s resilience against the extreme heterogeneity and noise of real-world cyberspace.

Large-Scale Asset Discovery. As illustrated in Fig. 6, *NS-FirmID* successfully identified 129,268 devices in total, extracting valid firmware versions from **58,451** of them. This represents a remarkable **211×** broader identification coverage than Nmap (which extracted a mere 277 versions) on the identical dataset. The results expose a highly fragmented IoT ecosystem, uncovering 25,176 firmware version strings spanning across **3,204 distinct brands**. While market leaders

like Dahua and Cisco exhibit standardized versioning (e.g., 2.421.000000.11.r and 12.4(1c)), a massive long tail of niche vendors employs highly chaotic formats, including dates, commit hashes, and custom alphanumeric codes. *NS-FirmID*'s capability to systematically structure this unstructured noise yields a high-resolution map of global firmware deployments.

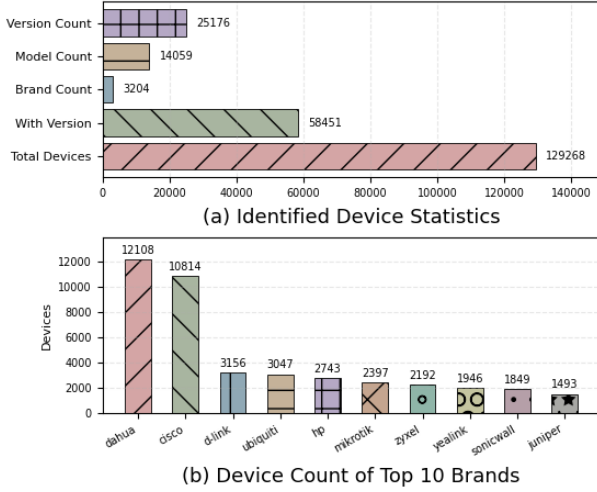


Fig. 6. In-the-wild identification results. *NS-FirmID* effectively maps a highly fragmented IoT ecosystem, extracting 58,451 valid firmware versions across 3,204 brands while highlighting the dominance of market leaders.

Detailed Vulnerability Landscape Analysis. The primary advantage of fine-grained firmware identification is its ability to accurately map individual devices to specific Common Vulnerabilities and Exposures (CVEs) [35]. To assess real-world exposure, we cross-referenced our extracted firmware versions against a consolidated vulnerability repository crawled from the National Vulnerability Database (NVD) [3]. Specifically, we systematically transformed the extracted device attributes into standard CPE URIs. By aligning these constructed CPEs with the vulnerable configuration nodes documented in the NVD—accounting for both exact version matches and inclusive vulnerable version ranges—we established reliable device-to-vulnerability linkages. In total, *NS-FirmID* mapped 44,131 distinct version-vulnerability correspondences. We conduct an in-depth analysis of this threat landscape:

(1) Vulnerability Severity Distribution. As illustrated in Fig. 7, the severity distribution of the 44,131 mapped vulnerabilities reveals an alarming reality. 27.4% of the identified CVE correspondences are classified as Critical, and an overwhelming 49.4% are rated as High severity. This indicates that severe security risks (comprising 76.8% of the total mappings) are deeply embedded within the fragmented IoT ecosystem. Such critical, high-resolution threat intelligence is entirely unattainable through traditional, coarse-grained brand or model identification, empirically proving the practical necessity of *NS-FirmID*'s fine-grained version extraction.

(2) Brand-Level Vulnerability Distribution. Fig. 8 visualizes the distribution of mapped CVE correspondences across major

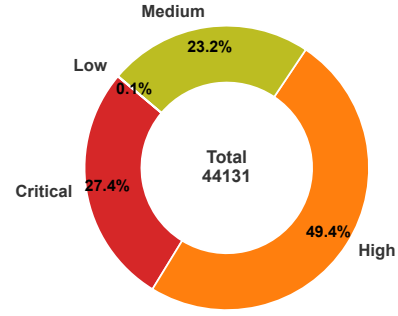


Fig. 7. Severity distribution of the 44,131 identified version-vulnerability correspondences. An alarming 76.8% of the mapped CVEs are classified as High or Critical severity, exposing severe security blind spots in the wild.

vendors. The distribution is heavily skewed toward legacy vendors with extensive product histories and massive installed bases. Notably, D-Link and Cisco collectively account for over 19,000 matched vulnerabilities. This aligns with their pervasive market presence and the widespread, continued operation of legacy devices well beyond their end-of-life cycles. Furthermore, major surveillance manufacturers, conspicuously Dahua (7,302 CVEs) and Axis (3,570 CVEs), exhibit substantial vulnerability footprints. These findings underscore the chronic exposure of Internet-facing IoT infrastructure, where cumbersome manual update processes and deprecated vendor support leave vast swaths of devices perpetually unpatched.

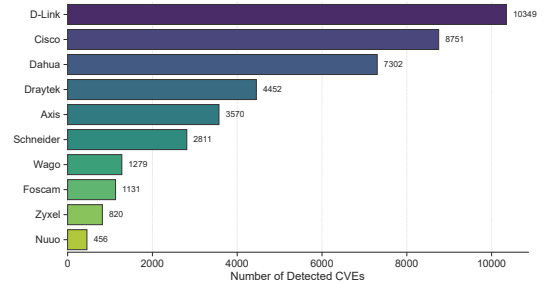


Fig. 8. Top 10 brands by CVE correspondences. The distribution is skewed, with legacy networking (D-Link, Cisco) and surveillance (Dahua) vendors dominating the vulnerability landscape due to prolonged EoL operations.

(3) High-Risk Vulnerability Concentration. To assess threats with immediate operational impact, Fig. 9 isolates exclusively Critical and High-severity CVEs. Contrasting this with the total-volume distribution reveals a shifting threat landscape: while D-Link dominates the overall CVE count, Cisco actually surpasses it in high-risk exposure, recording the highest count of severe vulnerabilities (7,338). This indicates a disproportionately high density of critical flaws deeply embedded within Cisco's legacy deployments. The pervasive persistence of these high-severity CVEs across major vendors exposes systemic failures in IoT firmware maintenance. Ultimately, this underscores the necessity of precise version identification, a core capability of *NS-FirmID*, to accurately distinguish genuinely vulnerable, unpatched assets from safely updated ones.

(4) Prevalence of Technical Vectors. Fig. 10 illustrates

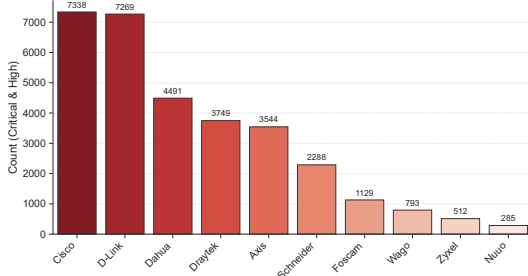


Fig. 9. Top 10 brands by High and Critical severity CVEs. Cisco surpasses D-Link in high-risk exposure, revealing a disproportionate density of severe vulnerabilities in its legacy equipment.

the distribution of Common Weakness Enumerations (CWEs) pervading the identified assets. The top three categories, namely Improper Restriction of Operations within Buffer Bounds (e.g., CVE-2019-9676 [36]), Out-of-bounds Write (e.g., CVE-2021-22791 [37]), and OS Command Injection (e.g., CVE-2023-20007 [38]), dominate the threat landscape. These patterns stem from a continued reliance on memory-unsafe legacy C/C++ codebases and the widespread absence of modern exploit mitigations in embedded firmware. Furthermore, the prominence of Hard-coded Credentials and Improper Authentication reflects persistent architectural deficiencies in access control. Unlike web-based threats that often cluster around client-side flaws, our measurements demonstrate that IoT firmware is primarily plagued by low-level, high-impact vulnerabilities. These flaws are particularly devastating, as they frequently provide direct primitives for unauthenticated remote code execution and full device compromise.

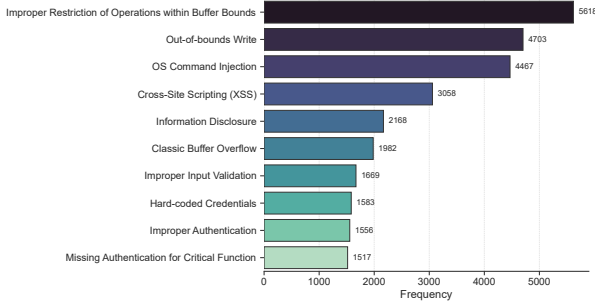


Fig. 10. Top 10 Common Weakness Enumerations (CWEs) among identified assets. Memory-related flaws and command injections dominate, reflecting the high prevalence of Remote Code Execution primitives in the wild.

The empirical findings in Figs. 7–10 underscore the practical utility of *NS-FirmID*. Conventional identification limited to brands or models fails to distinguish legacy deployments from updated ones, effectively obscuring actual risk. By extracting precise firmware versions, our method enables direct association between assets and the vulnerabilities affecting them. This precision transforms general enumeration into actionable intelligence, allowing defenders to prioritize the 27.4% of mapped vulnerabilities classified as Critical. Ultimately, *NS-FirmID* bridges the gap between simple network discovery and a comprehensive understanding of operational risk.

Answer to RQ4: *NS-FirmID* identifies $211\times$ more versions than Nmap across 150,000 banners, discovering 25,176 versions across 3,204 brands. This visibility enables mapping 44,131 vulnerability correspondences where 27.4% are Critical severity, proving its practicality for internet-wide measurement. The downstream vulnerability mapping underscores the value of *NS-FirmID* in cyberspace security, establishing it as a scalable solution to transform noisy network artifacts into actionable risk intelligence.

G. End-to-End Efficiency and Scalability Analysis

To evaluate operational feasibility, we benchmarked *NS-FirmID* against rule-based methods and general LLMs on a single NVIDIA A100 GPU (Table V). Nmap processes samples rapidly at 7.88 ms each, but its rigid regular expressions fail on heterogeneous banners, yielding a 0.1% F1 score. In contrast, *NS-FirmID* sacrifices sub-second latency to achieve a 93% F1 score. With an average inference time of 345.54 ms per sample (roughly 10,400 samples hourly per A100), the framework remains highly practical for asynchronous internet-scale auditing. The results also validate our local distillation strategy: the 7B student model operates over $60\times$ faster than the cloud-based DeepSeek-R1 API, avoiding both data privacy risks and token costs. Finally, while the Validator and Discriminator introduce a 130 ms overhead compared to the standalone Extractor, this neuro-symbolic feedback loop yields a 10.8% F1 improvement, making the multi-agent overhead acceptable for high-fidelity measurements.

H. Sensitivity to Heterogeneity and Adversarial Interference

Real-world IoT banners exhibit extreme format diversity. To quantify this, we classified the dataset into Low, Medium, and High heterogeneity levels using a weighted scoring scheme across five metrics: Shannon Entropy, Length, Word Count, Digit Ratio, and Structure Score. We applied a percentile-based threshold (33rd and 66th percentiles) and assigned the highest weight to Shannon Entropy to better capture information complexity. As Fig. 11(a) and (c) show, the High group reflects the structural chaos typical of long-tail ecosystems. Despite this variance, *NS-FirmID* maintains stable identification accuracies of 92.5%, 95.0%, and 92.3% across groups (Fig. 11b).

The Pearson correlation heatmap (Fig. 11d) shows processing time correlates strongly with banner length ($r = 0.934, p < 0.05$) because longer inputs require extended reasoning paths. However, correlations between complexity metrics and identification accuracy remain very low. This indicates that the DA’s ambiguity rejection mechanism effectively absorbs semantic noise without degrading performance.

To evaluate resilience against adversarial interference and decoy injection, we analyzed performance across varying obfuscation levels. We define Interference Level as the count of confounding strings (e.g., component version, build dates, OS kernels) within a banner. As Fig. 12 shows, *NS-FirmID* sustains over 95% accuracy under high interference of up to

TABLE V
END-TO-END PERFORMANCE, EFFICIENCY, AND SCALABILITY COMPARISON FOR FIRMWARE IDENTIFICATION.

Category	Model	Deployment	Parameters	Latency (ms)	Throughput (samples/s)	Hourly Capacity	F1 Score
<i>Rule-based</i>	Nmap	Local	-	7.88	126.93	~456,900	0.001
<i>Proposed (Ours)</i>	SFT-Qwen2.5-7B (EA)	Local	7B	215.62	4.64	~16,700	0.822
	NS-FirmID (Full Pipeline)	Local	7B	345.54	2.89	~10,400	0.930
<i>General Models</i>	Qwen2.5-7B	Local	7B	302.33	3.31	~11,900	0.784
	Llama-3-8B	Local	8B	571.98	1.75	~6,300	0.660
	Qwen-14B	Local	14B	434.97	2.30	~8,200	0.670
<i>Teacher Model</i>	DeepSeek-R1-671B	Cloud API	671B	21540.00	0.046	~165	0.855

30 strings, declining to 78.18% only under extreme obfuscation. This proves the Discriminator safely aborts ambiguous extractions to mitigate spoofing.

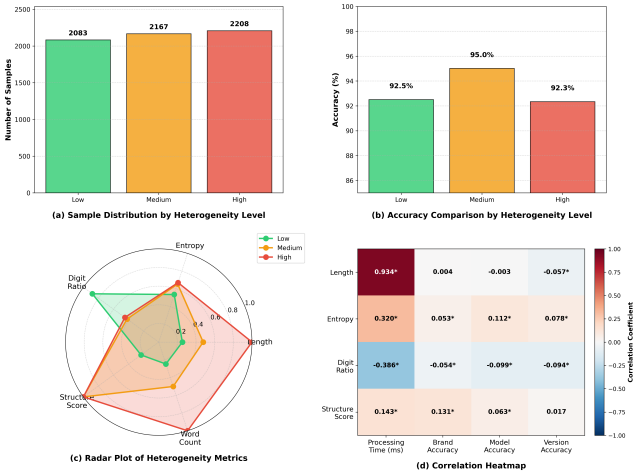


Fig. 11. Comprehensive evaluation of banner heterogeneity. (a) Sample distribution. (b) Accuracy stability. (c) Radar plot. (d) Correlation heatmap (* indicates $p < 0.05$).

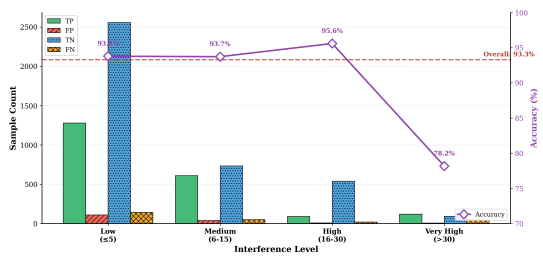


Fig. 12. Result distribution and accuracy under different interference levels (count of confounding strings). Bars show TP/FP/TN/FN; line indicates accuracy. Red dashed line marks overall accuracy (93.3%).

V. LIMITATIONS

We acknowledge several limitations that define the boundaries of our current approach:

Dependency on Banner Visibility: *NS-FirmID* requires explicit or semi-explicit version strings in banners, failing against banner suppression or complete obfuscation. Under perfect semantic spoofing via injected historical or syntactically valid

future versions, the framework extracts the spoofed artifact but lacks behavioral verification. This reflects the inherent trade-off between lightweight active probing and intrusive vulnerability scanning. Developing device behavioral fingerprinting to detect such spoofing remains our future work.

Inference Latency vs. Rule-Based Speed: Although the underlying SLM (7B parameters) is optimized for efficiency, the multi-agent inference process, which involves reasoning generation and multi-dimensional feature extraction, incurs a higher computational cost than static regex matching (e.g., Nmap). Therefore, *NS-FirmID* is best deployed as a second-stage analyzer for complex, long-tail assets where traditional rule-based tools fail, rather than a line-rate packet inspector.

Knowledge Base Completeness: The VA’s effectiveness in the secondary pass inherently depends on the coverage of our knowledge base. For entirely novel vendors absent from our records, the system cannot leverage historical naming patterns and degrades to standard zero-shot extraction. While the semantic reasoning of the EA mitigates this, reliability naturally decreases compared to established vendors.

VI. CONCLUSION

Firmware version identification remains a cornerstone of cyberspace defense, historically hindered by vendor heterogeneity and extreme network noise. We present **NS-FirmID**, a neuro-symbolic multi-agent framework designed for this challenge. By synergizing the semantic flexibility of language models with symbolic knowledge verification, and orchestrating Extractor, Validator, and Discriminator agents, we establish a robust “Extract-Verify-Decide” paradigm.

Evaluations demonstrate *NS-FirmID* reduces the false positive rate by 17.1% while achieving a 93% F1-score, surpassing the DeepSeek-R1 teacher using only 1% of its parameters. Crucially, the system proves highly resilient: it sustains $> 95\%$ accuracy against severe structural heterogeneity and adversarial decoys, while delivering a throughput of 10,400 samples hourly per A100 GPU. Deployed in the wild, the framework achieved coverage $211\times$ broader than Nmap. This fine-grained visibility revealed that 27.4% of the mapped vulnerabilities pose Critical-severity risks. Ultimately, this work validates neuro-symbolic AI as a scalable and reliable foundation for next-generation cyberspace mapping and vulnerability management.

REFERENCES

- [1] Frank Ebberts, “A large-scale analysis of iot firmware version distribution in the wild,” *IEEE Transactions on Software Engineering*, vol. 49, no. 2, pp. 816–830, 2022.
- [2] Zhenhao Tian, Yi He, Nuo Zhang, Qixiao Lin, Hetian Shi, Jianwei Zhuge, Jian Mao, and Deliang Chang, “Blmprobe: Enhancing internet-connected device discovery by automated device labeling and label migration,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 7227–7242, 2025.
- [3] “National vulnerability database,” Available: <https://nvd.nist.gov/>, Accessed: 2025-10-12.
- [4] “Common platform enumeration,” Available: <https://nvd.nist.gov/products/cpe>, Accessed: 2025-10-12.
- [5] Qiang Li, Xuan Feng, Haining Wang, Zhi Li, and Limin Sun, “Towards fine-grained fingerprinting of firmware in online embedded devices,” in *2018 IEEE Conference on Computer Communications, INFOCOM 2018, Honolulu, HI, USA, April 16-19, 2018*, pp. 2537–2545, IEEE.
- [6] Qiang Li, Xuan Feng, Haining Wang, and Limin Sun, “Automatically discovering surveillance devices in the cyberspace,” in *Proceedings of the 8th ACM on Multimedia Systems Conference*, 2017, pp. 331–342.
- [7] Xu Wang, Yucheng Wang, Xuan Feng, Hongsong Zhu, Limin Sun, and Yuchi Zou, “Iottracker: An enhanced engine for discovering internet-of-things devices,” in *2019 IEEE 20th International Symposium on “A World of Wireless, Mobile and Multimedia Networks”(WoWMoM)*, IEEE, 2019, pp. 1–9.
- [8] Kai Yang, Yueyang Zhang, Xiaodong Lin, Zhi Li, and Limin Sun, “Characterizing heterogeneous internet of things devices at internet scale using semantic extraction,” *IEEE Internet of Things Journal*, vol. 9, no. 7, pp. 5434–5446, 2021.
- [9] Xuan Feng, Qiang Li, Haining Wang, and Limin Sun, “Acquisitional rule-based engine for discovering {Internet-of-Things} devices,” in *27th USENIX security symposium (USENIX Security 18)*, 2018, pp. 327–341.
- [10] Andrei Costin, Apostolis Zarras, and Aurélien Francillon, “Towards automated classification of firmware images and identification of embedded devices,” in *ICT Systems Security and Privacy Protection: 32nd IFIP TC 11 International Conference, SEC 2017, Rome, Italy, May 29-31, 2017, Proceedings 32*, Springer, 2017, pp. 233–247.
- [11] Weidong Zhang, Yu Chen, Hong Li, Zhi Li, and Limin Sun, “PAN-DORA: A scalable and efficient scheme to extract version of binaries in iot firmwares,” in *2018 IEEE International Conference on Communications, ICC 2018, Kansas City, MO, USA, May 20-24, 2018*, 2018, pp. 1–6, IEEE.
- [12] Qiang Li, Dawei Tan, Xin Ge, Haining Wang, Zhi Li, and Jiqiang Liu, “Understanding security risks of embedded devices through fine-grained firmware fingerprinting,” *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 6, pp. 4099–4112, 2022.
- [13] Wei Xie, Chao Zhang, Pengfei Wang, Zhenhua Wang, and Qiang Yang, “Argus: assessing unpatched vulnerable devices on the internet via efficient firmware recognition,” in *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, 2021, pp. 421–431.
- [14] Gordon Fyodor Lyon, *Nmap network scanning: The official Nmap project guide to network discovery and security scanning*, Insecure, 2009.
- [15] Shodan, “The search engine for internet-connected devices,” Available: <https://www.shodan.io/>.
- [16] Armin Sarabi, Tongxin Yin, and Mingyan Liu, “An llm-based framework for fingerprinting internet-connected devices,” in *Proceedings of the 2023 ACM on Internet Measurement Conference*, New York, NY, USA, 2023, IMC ’23, p. 478–484, Association for Computing Machinery.
- [17] Rameen Mahmood, Tousif Ahmed, Sai Teja Peddinti, and Danny Yuxing Huang, “Large language models for real-world iot device identification,” *arXiv preprint arXiv:2510.13817*, 2025.
- [18] Kyle Mahowald, Anna A Ivanova, Idan A Blank, Nancy Kanwisher, Joshua B Tenenbaum, and Evelina Fedorenko, “Dissociating language and thought in large language models,” *Trends in cognitive sciences*, vol. 28, no. 6, pp. 517–540, 2024.
- [19] Zhangyue Yin, Qiushi Sun, Qipeng Guo, Jiawen Wu, Xipeng Qiu, and Xuanjing Huang, “Do large language models know what they don’t know?,” *arXiv preprint arXiv:2305.18153*, 2023.
- [20] Qwen Team, “Qwen2.5: A party of foundation models,” September 2024.
- [21] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al., “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [22] Zhaoteng Yan, Zhi Li, Hong Li, Shouguo Yang, Hongsong Zhu, and Limin Sun, “Internet-scale fingerprinting the reusing and rebranding iot devices in the cyberspace,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3890–3909, 2022.
- [23] Shangfeng Wan, Qiang Li, Haining Wang, Hong Li, and Limin Sun, “Devtag: A benchmark for fingerprinting iot devices,” *IEEE Internet of Things Journal*, vol. 10, no. 7, pp. 6388–6399, 2023.
- [24] “Ztag: An utility for annotating raw scan data with additional metadata,” Available: <http://github.com/zmap/ztag>, Accessed: 2025-11-12. [Online].
- [25] Censys, “The search engine for internet-connected devices,” Available: <https://search.censys.io/>.
- [26] Chen Liang, Bo Yu, Wei Xie, Baosheng Wang, and Wei Peng, “Fine-grained identification for large-scale iot devices: A smart probe-scheduling approach based on information feedback,” *Applied Sciences*, vol. 12, no. 16, 2022.
- [27] Dan Yu, Lilong Zhang, Yongle Chen, Yao Ma, and Junjie Chen, “Large-scale iot devices firmware identification based on weak password,” *IEEE Access*, vol. 8, pp. 7981–7992, 2020.
- [28] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [29] Tianqi Chen, “Xgboost: A scalable tree boosting system,” *Cornell University*, 2016.
- [30] Cydar, “Internet of things device discovery and identification system,” Available: <https://cydar.cn/>.
- [31] Internet Assigned Numbers Authority, “Internet Assigned Numbers Authority (IANA),” Available: <http://www.iana.org/>.
- [32] Fofa, “The search engine for internet-connected devices,” Available: <https://en.fofa.info/>.
- [33] Qwen Team, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [34] AI@Meta, “Llama 3 model card,” 2024.
- [35] “Common vulnerabilities and exposures,” Available: <https://cve.mitre.org/>, Accessed: 2025-10-12.
- [36] Common Vulnerabilities and Exposures, “CVE-2019-9676,” Available: <https://nvd.nist.gov/vuln/detail/CVE-2019-9676>, 2019.
- [37] Common Vulnerabilities and Exposures, “CVE-2021-22791,” Available: <https://nvd.nist.gov/vuln/detail/CVE-2021-22791>, 2023.
- [38] Common Vulnerabilities and Exposures, “CVE-2023-20007,” Available: <https://nvd.nist.gov/vuln/detail/CVE-2023-20007>, 2023.